Hackathon IA — La Banque Postale × CGI × ENSEIRB-MATMECA — Sujet 1

Guide de la solution

Ce qu'on fait, comment, et dans quel code

Yahya (IA) — Sohayb (génie logiciel) — Younes (calcul intensif) — mardi 27 octobre 2026

État du dépôt : 171 tests et 10 scénarios au vert.

1 La solution

1.1 En une phrase

Un client écrit à sa banque ; notre assistant **comprend** la demande, **pose les questions qui manquent**, **décide** avec des règles bancaires publiques, **agit** si besoin (opposition, virement), puis **répond** en français — en n'utilisant un LLM qu'au dernier moment, et seulement si c'est utile.

1.2 Le principe : une cascade

Chaque étage ne passe la main au suivant que s'il ne sait pas faire lui-même. Le moins cher d'abord, le LLM en dernier. C'est l'image donnée par l'organisateur : le vélo pour aller chercher le pain, la voiture pour le week-end.

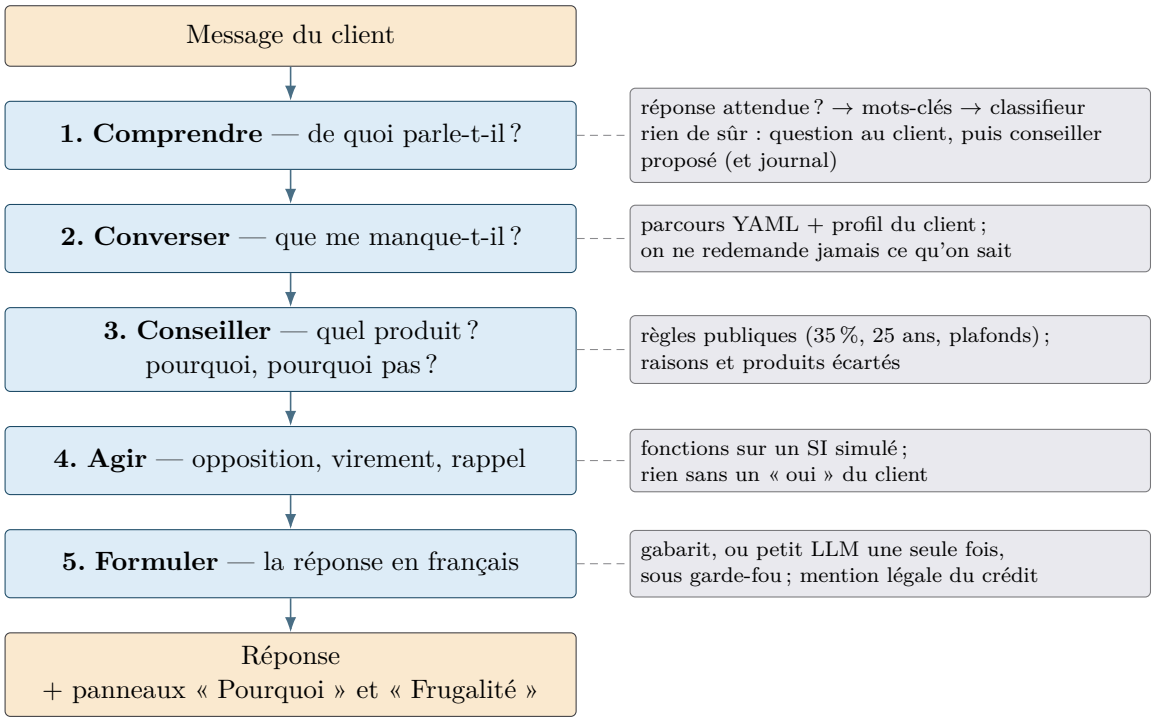


Figure 1 – La cascade. Selon la demande, l'étage 3 ou l'étage 4 est sauté.

1.3 Un exemple complet : Léa veut un appartement

Message	Ce qui se passe
« Je veux acheter un appartement »	**Comprendre** : les mots-clés « acheter un appartement » donnent `credit_immobilier`, en moins d'une milliseconde, sans modèle. **Converser** : le parcours `credit_immobilier.yaml` demande 4 informations ; revenus et crédits en cours sont *dans son profil*, donc confirmés ; il manque le prix et l'apport. **Formuler** : « C'est un beau projet. J'ai noté vos revenus de 2 900 € nets par mois. Quel est le prix du bien… ? » (gabarit, sans LLM).
« Autour de 250 000, et je peux mettre les 25 000 »	**Comprendre** : une question était en attente, donc c'est *la réponse* ; les montants sont extraits par règles. **Conseiller** : règles HCSF au taux moyen Banque de France de 3,32 % : mensualité de 1 197 €, soit **41,3 % des revenus, au-delà des 35 %** : projet non retenu, sans refus bancaire ; crédit sur 30 ans écarté (25 ans maximum) ; budget indicatif d'environ 215 000 € ; hypothèses et sources affichées ; rendez-vous conseiller proposé. **Formuler** : explication, mention légale, proposition de rappel.
« Et avec un bien à 200 000 ? »	Compris comme une **correction** : le prix est mis à jour et le conseil recalculé : 32,2 %, orientation vers un conseiller.

Pendant ce temps, l'interface affiche l'intention et comment elle a été comprise, les règles appliquées, les produits proposés et écartés, et le temps de chaque étape — sans aucun appel au LLM.

1.4 Chaque étage : quoi, comment, pourquoi

Étage	Comment	Pourquoi
1. Comprendre (Younes)	(1) réponse à la question en cours ; (2) mots-clés, qui ne concluent que s'ils sont sûrs ; (3) classifieur `multilingual-e5-small` + régression logistique, entraîné sur Banking77 **en anglais, sans traduction** (à faire) ; (4) sinon : question au client, puis conseiller proposé et message noté dans un journal. Pas de gros modèle par défaut.	Comprendre ne demande pas de LLM : rapide, presque gratuit, mesurable.
2. Converser (Yahya)	Un parcours par situation, en YAML (quoi demander, où le trouver, quoi faire à la fin) ; un automate les lit. Cas limites : « je ne sais pas », correction, abandon.	Déterministe, lisible, testable, sans LLM.
3. Conseiller (Sohayb)	Règles publiques sourcées : taux d'effort $\leq 35\,\%$, durée ≤ 25 ans, plafonds des livrets, âges. Chaque proposition a ses raisons, chaque produit écarté la sienne.	Une décision financière doit être juste, reproductible et explicable.
4. Agir (Sohayb)	5 actions sur un SI simulé : opposition, virement, explication des frais, rappel conseiller, lecture du profil. Tout effet de bord exige un « oui ».	Sécurité du client.
5. Formuler (Yahya)	Gabarits par défaut. Si activé, le petit LLM (`ministral-3:3b`) reformule un conseil, une seule fois ; le garde-fou refuse tout nombre absent du gabarit. Mode « mots simples » pour les clients âgés ou peu à l'aise.	Exact et gratuit d'abord ; le LLM seulement pour le naturel.
Mesurer (Younes)	Chaque étage est chronométré, les appels au LLM comptés ; CodeCarbon à venir pour comparer A (tout LLM), B (notre cascade) et C (sans LLM).	La frugalité vaut 20 % de la note.

1.5 Les données

Donnée	Rôle
Banking77 (anglais, 77 intentions)	Entraîner le classifieur, **sans traduction**
Bank Marketing (45 211 clients)	Ancrer les 8 clients de démo ; futur modèle d'appétence
`clients.json`	Inès, Marc, Sophie, Paul, Karim, Julie, Léa, Lucas
`catalogue.json`	Produits et paramètres réglementaires, avec leur source
`glossaire.json`	Explications du jargon bancaire

1.6 Où on en est

Fait : toute la chaîne, les 9 parcours, les règles, les actions, l'interface, le mode « mots simples », le journal des incompris, le branchement réel du LLM, la démo de secours et le support de soutenance — 171 tests et 10 scénarios au vert. **À faire** : classifieur (Younes) ; choix du LLM sur un poste de la salle (Yahya) ; appétence et sources du catalogue (Sohayb) ; mesures d'énergie, vidéo, diapositives.

En 30 secondes. *Notre assistant comprend avec de simples mots-clés ou un petit modèle de 118 millions de paramètres, décide avec des règles bancaires que le client peut lire, et n'appelle un LLM qu'une fois, à la fin, pour mettre en mots ce qui est déjà calculé. Chaque conseil dit pourquoi, et pourquoi pas. Et on l'a mesuré face à une version où un gros LLM fait tout.*

2 Le code

2.1 Les technologies

Besoin	Technologie	Remarque
Langage	Python 3.10+	Tout le projet
Interface	Streamlit	Une page web écrite en Python
Parcours de conversation	YAML (PyYAML)	Fichiers lisibles par un humain
Données	JSON	Clients, catalogue, glossaire
LLM local	Ollama (poste de la salle) ou vLLM (DeepEirb, API compatible OpenAI)	Appelés avec `urllib` (bibliothèque standard) ; serveur choisi par rôle ; désactivé par défaut
Tests	pytest + `streamlit.testing`	L'interface est testée sans navigateur
À venir (Younes)	sentence-transformers, scikit-learn, CodeCarbon	Classifieur et mesure d'énergie

Dépendances installées aujourd'hui : `streamlit`, `pyyaml`, `pytest`. Tout le reste vient de la bibliothèque standard de Python.

2.2 L'arborescence

```
hackathon-IA/
|-- app.py                  Interface Streamlit                                  (Yahya)
|-- src/
|   |-- contrats.py         La langue commune : structures echangees entre modules
|   |-- taxonomie.py        14 familles, 85 intentions (77 Banking77 + 8 a nous)
|   |-- assistant.py        reply() : l'orchestrateur, un tour de conversation   (Yahya)
|   |-- llm.py              Appel a Ollama (desactive par defaut)                (Yahya)
|   |-- texte.py            Normaliser un texte, ecrire 1 197 EUR et 41,3 %
```

```
|   |-- incompris.py        Journal des messages non compris                    (Yahya)
|   |-- comprendre/         ETAGE 1                                             (Younes)
|   |   |-- __init__.py     comprendre() : reponse attendue -> mots-cles -> classifieur
|   |   |-- mots_cles.py    Lexique par intention + regle "sur ou pas"
|   |   |-- entites.py      Regex : montants, durees, liens, oui/non
|   |   |-- classifieur.py  BOUCHON : e5-small + regression logistique, a entrainer
|   |   `-- escalade.py     Appel au 14B en dernier recours (desactive par defaut)
|   |-- converser/          ETAGES 2 et 5                                       (Yahya)
|   |   |-- dialogue.py     L'automate : lit les parcours, decide quoi faire
|   |   |-- parcours/*.yaml 9 parcours, un par situation du sujet
|   |   |-- formuler.py     Gabarits, LLM eventuel, mode "mots simples"
|   |   |-- garde_fou.py    Refuse les nombres inventes et les promesses
|   |   `-- glossaire.py    Explication d'un terme bancaire
|   |-- conseiller/         ETAGES 3 et 4                                       (Sohayb)
|   |   |-- clients.py      Charge les profils (adaptateur de la base clients)
|   |   |-- catalogue.py    Charge produits et parametres sources
|   |   |-- regles.py       Moteur de regles : credit, epargne, pret etudiant
|   |   `-- outils.py       Les 5 actions + confirmation obligatoire + journal
|   `-- mesure/__init__.py  Chronometrage de chaque etape                       (Younes)
|-- data/raw/               Banking77, Bank Marketing (bruts)
|-- data/processed/         clients.json, catalogue.json, glossaire.json
|-- eval/scenarios/*.yaml   9 conversations de reference
|-- eval/run_scenarios.py   Les rejoue et verifie les attentes
|-- eval/bench_formulation.py   Choisit le LLM en descendant en taille
`-- tests/                  171 tests pytest
```

2.3 Le contrat : comment les modules se parlent

Les modules ne s'appellent pas n'importe comment : ils échangent des **dataclasses** définies une seule fois dans `src/contrats.py`. Chacun peut ainsi avancer sans attendre les autres.

Structure	Rendue par	Contenu
`Comprehension`	Comprendre	famille, intention, confiance, niveau (mots-clés, classifieur, réponse attendue, escalade), entités (montants, durée, lien, objet), alternatives, insatisfaction
`Conseil`	Conseiller	propositions (`Proposition` : produit, raisons, sources, score), écartés (`Ecart` : produit, raison, règle — le « pourquoi pas »), alertes, faits (les seuls nombres autorisés dans la réponse)
`Action`	Agir	outil, paramètres, confirmée (faux par défaut), résultat
`Etape`	Mesurer	une ligne du panneau Frugalité : nom, modèle, latence, tokens, appel au LLM
`Reponse`	`reply()`	texte, compréhension, conseil, action, question posée, escalade, trace
`EtatConversation`	`reply()`	la mémoire entre deux messages : parcours, information attendue, informations recueillies, action en attente, mots simples, historique

2.4 Le chemin d'un message dans le code

Tout part de `reply(message, client_id, etat)`, dans `src/assistant.py` :

```
profil = clients.profil(client_id)                      # conseiller/clients.py -> clients.json

# 0. Une action attend un "oui" ?          -> outils.executer(action)   (conseiller/outils.py)

# 1. Comprendre                                                (comprendre/__init__.py)
comprehension = comprendre(message, etat)
#    etat.info_attendue ?            -> c'est la reponse a la question posee
#    mots_cles.detecter(message)     -> sur ? on s'arrete
```

```
#    classifieur.predire(message)    -> au-dessus de tau ? on s'arrete
#    + entites.extraire(message)     -> montants, durees...
#    pas compris -> Decision("clarifier"), puis Decision("orienter") + incompris.noter()

# 2. Converser                                                  (converser/dialogue.py)
decision = avancer(message, comprehension, etat, profil)
#    lit parcours/<intention>.yaml, remplit etat.slots (profil, puis message)
#    -> "question" | "conseil" | "action" | "transmettre" | "fiche" | "abandon"

# 3. Conseiller (si conseil)                                    (conseiller/regles.py)
conseil = conseiller(etat.parcours, profil, etat.slots)

# 4. Agir (si action) : sans effet de bord -> execute ; sinon -> attend un "oui"

# 5. Formuler                                                   (converser/formuler.py)
texte = formuler(decision, conseil, action, profil, etat, message)
#    gabarit -> [LLM + garde-fou si ASSISTANT_LLM=1 et conseil] -> mots simples -> mention legale

return Reponse(texte, comprehension, conseil, action, ..., trace)   # chaque etape chronometree
```

2.5 Les briques une par une

`mots_cles.py` — le filtre d'entrée. Un dictionnaire `{intention: (expressions)}`, cherché dans le message normalisé (« VOLÉ MA CB » devient « vole ma cb »). Règle « sûr » : une seule intention trouvée, et soit 12 mots au plus, soit au moins 2 expressions.

`entites.py` — les informations du message. Des expressions régulières : « 250 000 », « 250k » et « 1 900 € » deviennent des montants ; « 10 ans » donne 120 mois ; « mon fils » donne le lien ; « oui » et « non » pour les confirmations. Durées et heures sont retirées avant de chercher les montants.

`parcours/*.yaml` et `dialogue.py` — l'automate. Un parcours, par exemple `credit_immobilier.yaml` (extrait) :

```
infos:
  revenus_nets:
    type: montant
    source: revenus_nets                    # lu dans le profil...
    confirmer: "J'ai note vos revenus de {valeur} nets par mois."   # ...et confirme
  prix_bien:
    type: montant
    indices: [prix, bien]                   # pour les corrections
    question: "Quel est le prix du bien que vous visez ?"
fin: {type: conseil, mention_legale: credit, proposer: transmettre_conseiller}
```

`avancer()` remplit les informations depuis le profil (`source`) puis depuis le message (selon le `type`), pose la première question qui manque, sinon déclenche la fin. Clés spéciales : `si_inconnu` (valeur prise sur « je ne sais pas »), `si_non` (valeur prise sur « non »), `indices` (à quelle information va un montant lors d'une correction).

`regles.py` — le moteur de conseil. Une fonction par parcours. Le crédit immobilier applique la formule d'annuité, capacité $= m \times \frac{1-(1+i)^{-n}}{i}$, avec les paramètres du catalogue (35 %, 25 ans, frais de notaire, taux indicatif). Il rend un `Conseil` avec raisons, écarts et identifiant de règle (`HCSF-taux-effort`).

`outils.py` — les actions. `OUTILS` déclare chaque outil et s'il a un effet de bord. `executer()` refuse un outil à effet de bord non confirmé (exception `ConfirmationRequise`). Le SI est simulé sur une copie

du profil ; chaque action est notée dans un journal.

`formuler.py` et `garde_fou.py` — la réponse. Un gabarit par type de décision et par outil. Si `ASSISTANT_LLM=1` et que c'est un conseil, un seul appel à Ollama reformule le gabarit ; `verifier()` compare les nombres de la reformulation à ceux du gabarit et refuse les promesses (« accordé », « garanti »). Il refuse aussi les chiffres *oubliés* et les réponses tronquées : un test réel avec `gemma3:270m` répondait « Oui, je suis prêt à reformuler le message » sans le conseil. Au moindre écart, le gabarit est gardé. La même fonction `reformuler()` sert à l'assistant et au banc de formulation.

`glossaire.py` — le jargon. Recherche exacte du terme, puis, pour une question de définition, recherche approchante (`difflib`) qui rattrape les fautes : « TEAG » donne TAEG. Pas de modèle : une vingtaine de fiches écrites et relues.

`app.py` — l'interface. Barre latérale : choix du client (C01 à C08), messages de démo cliquables, interrupteurs « Mots simples » et « Grand texte ». Trois zones : la conversation, « Pourquoi » (raisons, écarts, sources) et « Frugalité » (une ligne par étape, totaux de la conversation). La barre latérale affiche l'état *réel* du LLM (serveur interrogé, modèle préchargé), pas seulement la configuration.

2.6 La configuration

Variable	Défaut	Effet
`ASSISTANT_LLM`	`0`	`1` : le petit LLM reformule les conseils
`ASSISTANT_ESCALADE`	`0`	`1` : appel au 14B si un message reste incompris
`LLM_API` (et `_FORMULATION`, `_ESCALADE`)	`ollama`	`ollama` ou `vllm`, par rôle
`VLLM_BASE_URL`	—	Serveur vLLM de DeepEirb (`.../v1`)
`OLLAMA_BASE_URL`	`localhost:11434`	Adresse d'Ollama
`OLLAMA_MODEL`	`ministral-3:3b`	Modèle de formulation
`OLLAMA_MODEL_ESCALADE`	`ministral-3:14b`	Modèle d'escalade
`INCOMPRIS_FICHIER`	`eval/incompris.jsonl`	Où écrire le journal

2.7 Lancer, tester, évaluer

```
streamlit run app.py                              # l'interface
python -m pytest -q                               # 171 tests
python -m eval.run_scenarios                      # 9 conversations de bout en bout
python -m eval.bench_formulation --situations     # le banc LLM (sans modele : les 30 cas)
python -m src.incompris                           # relire les messages non compris
python -m eval.demo_secours                       # rejoue la demo -> rapport/demo-secours.tex
python -m eval.tableau_formulation                # resultats du banc -> diapositive
```

Les tests vérifient chaque brique : la taxonomie couvre les 77 intentions officielles ; les mots-clés reconnaissent les situations des slides, et « carte » seul ne suffit pas ; le calcul HCSF donne 1 197 € et 41,3 % ; les actions refusent de s'exécuter sans confirmation ; le garde-fou refuse les nombres inventés ; l'interface démarre et répond.

Les scénarios rejouent les conversations des 8 clients et vérifient ce qui est demandé, ce qui ne doit *pas* l'être, les produits proposés et écartés, et deux règles partout : au plus un appel au LLM par tour, aucune action sans confirmation.